Purrr and a Bit of Dplyr

Statistical Computing, STA3005

Jan 21, 2026

Last chapter: Data frames and apply

- Data frames are a representation of the "classic" data table in R: rows are observations/cases, columns are variables/features
- Each column can be a different data type (but must be the same length)
- Factors represent a vector by categories (`levels`) and integer indices
- `subset()` : function for extracting rows of a data frame meeting a condition
- `split()` : function for splitting up rows of a data frame, according to a factor variable
- `apply()` : function for applying a given routine to rows or columns of a matrix or data frame
- `lapply()` : similar, but used for applying a routine to elements of a vector or list
- `sapply()` : similar, but will try to simplify the return type, in comparison to `lapply()`
- `tapply()` : function for applying a given routine to groups of elements in a vector or list, according to a factor variable

Part I: Three types of implicit iterations

Common iteration tasks

Here's a basic breakdown for common iteration tasks that we encounter in R: we iterate over

- elements of a list
- dimensions of an array (e.g., rows/columns of a matrix)
- subsets of a data frame induced by one or more factors

For simplicity, we usually implement the apply family of functions in base R: `lapply()`, `sapply()`, `apply()`, `tapply()`, etc, instead of using explicit iterations (`for` or `while` loop). Besides base R, we introduce another method— `tidyverse`.

Why do we look anywhere else?

When we apply `sapply()` to a mixed-type list containing numeric, character, and logical elements, `sapply()` could be risky for production code.

```
# Mixed-type list: numeric vector, character vector, logical matri
mixed_list <- list(nums = 1:5,
                   chars = letters[1:3],
                   bool_mat = matrix(sample(c(TRUE, FALSE), 6, rep
)

# lapply() returns a list
lapply_out <- lapply(mixed_list, function(x) length(x))
lapply_out
```

```
## $nums
## [1] 5
##
## $chars
## [1] 3
##
## $bool_mat
## [1] 6
```

```
# sapply() auto-simplifies (risky for mixed inputs)
sapply_out <- sapply(mixed_list, function(x) length(x))
```

```
sapply_out
```

```
##     nums    chars bool_mat
##        5        3        6
```

```
# Risk: If elements return different types, sapply() returns a lis
risky_sapply <- sapply(mixed_list, function(x) if (is.matrix(x)) d
risky_sapply
```

```
## $nums
## [1] 5
##
## $chars
## [1] 3
##
## $bool_mat
## [1] 2 3
```

In comparison, `tidyverse` offer better **consistency**

- With the apply family of functions, there are some inconsistencies in both the **interfaces** to the functions, as well as their **outputs**
- This can both slow down learning and also lead to inefficiencies in practice (frequent checking and post-processing of results)

However, the world isn't black-and-white: base R still has its advantages, and the best thing you can do is to be informed and well-versed in using all the major options!

Part II: `purrr`

`tidyverse` package

The `tidyverse` is a coherent collection of packages in R for data science (and `tidyverse` itself is actually a package that loads all its constituent packages). Packages include:

- **Data wrangling**: `dplyr`, `tidyr`, `readr`

- **Iteration**: `purrr`
- **Visualization**: `ggplot2`

We'll cover `purrr` and a bit of `dplyr` in the following. Next chapter we will do more `dplyr`, and some `tidyr`.

What is `purrr`?

`purrr` is a package that is part of the `tidyverse`. It offers a family of functions for iterating (mainly over lists) that can be seen as alternatives to base R's family of apply functions

- Compared to base R, they are more consistent

Below we install `tidyverse` which gives us the packages we need (`purrr` and `dplyr`).

```
install.packages("tidyverse")
```

Here, we load the `tidyverse` package.

```
library(tidyverse)
```

```
## ── Attaching core tidyverse packages ──────────────────────── t
## ✓ dplyr     1.1.4     ✓ readr     2.1.6
## ✓ forcats   1.0.1     ✓ stringr   1.6.0
## ✓ ggplot2   4.0.1     ✓ tibble    3.2.1
## ✓ lubridate 1.9.4     ✓ tidyr     1.3.2
## ✓ purrr     1.2.1
## ── Conflicts ────────────────────────────────────────── tidyver
## ✗ dplyr::filter() masks stats::filter()
## ✗ dplyr::lag()    masks stats::lag()
## ℹ Use the conflicted package (<http://conflicted.r-lib.org/>) t
```

Note: Loading the `tidyverse` package will cause namespace overlapping issues. In particular, `filter` functions in `dplyr` will mask the corresponding functions with the same names in `stats::filter`.

The map family

`purrr` offers a family of **map functions**, which allow you to apply a function across different chunks of data (primarily used with lists). Offers an alternative base R's apply functions. Summary of functions:

- `map()` : apply a function across elements of a list or vector
- `map_dbl()` , `map_lgl()` , `map_chr()` : same, but return a vector of a particular data type
- `map_dfr()` , `map_dfc()` : same, but return a data frame

`map()` : list in, list out

The `map()` function is an alternative to `lapply()` . It has the following simple form: `map(x, f)` , where `x` is a list or vector, and `f` is a function. It always returns a list

```
my.list = list(nums=seq(0.1,0.6,by=0.1), chars=letters[1:12],
               bools=sample(c(TRUE,FALSE), 6, replace=TRUE))
map(my.list, length)
```

```
## $nums
## [1] 6
##
## $chars
## [1] 12
##
## $bools
## [1] 6
```

```
# Base R is just as easy
lapply(my.list, length)
```

```
## $nums
## [1] 6
##
## $chars
## [1] 12
##
## $bools
## [1] 6
```

`map_dbl()` : list in, numeric out

The `map_dbl()` function is an alternative to `sapply()`. It has the form: `map_dbl(x, f)`, where `x` is a list or vector, and `f` is a function that returns a numeric value (when applied to each element of `x`)

Similarly:

- `map_int()` returns an integer vector
- `map_lgl()` returns a logical vector
- `map_chr()` returns a character vector

```
map_dbl(my.list, length)
```

```
##  nums chars bools
##     6    12     6
```

```
map_chr(my.list, length)
```

```
## Error in `map_chr()`:
## ℹ In index: 1.
## ℹ With name: nums.
## Caused by error:
## ! Can't coerce from an integer to a string.
```

```
# Base R is a bit more complicated
sapply(my.list, length)
```

```
##  nums chars bools
##     6    12     6
```

```
unlist(lapply(my.list, length))
```

```
##  nums chars bools
##     6    12     6
```

```
vapply(my.list, FUN=length, FUN.VALUE=numeric(1))
```

```
##  nums chars bools
##     6    12     6
```

Applying a custom function

As before (with the apply family), we can of course apply a custom function, and define it "on-the-fly"

Also, you need to install the package before using it

```
install.packages("repurrrsive")
```

`got_chars` contains the characters from the first five books in the Song of Ice and Fire series by George R. R. Martin.

```
library(repurrrsive)
class(got_chars)
```

```
## [1] "list"
```

```
class(got_chars[[1]])
```

```
## [1] "list"
```

```
names(got_chars[[1]])
```

```
##  [1] "url"         "id"          "name"        "gender"      "c
##  [6] "born"        "died"        "alive"       "titles"      "a
## [11] "father"      "mother"      "spouse"      "allegiances" "b
## [16] "povBooks"    "tvSeries"    "playedBy"
```

```
map_chr(got_chars, function(x) { return(x$name) })
```

```
##  [1] "Theon Greyjoy"      "Tyrion Lannister"   "Victarion Greyj
##  [4] "Will"               "Areo Hotah"         "Chett"
##  [7] "Cressen"            "Arianne Martell"    "Daenerys Targar
## [10] "Davos Seaworth"     "Arya Stark"         "Arys Oakheart"
## [13] "Asha Greyjoy"       "Barristan Selmy"    "Varamyr"
## [16] "Brandon Stark"      "Brienne of Tarth"   "Catelyn Stark"
## [19] "Cersei Lannister"   "Eddard Stark"       "Jaime Lannister
## [22] "Jon Connington"     "Jon Snow"           "Aeron Greyjoy"
## [25] "Kevan Lannister"    "Melisandre"         "Merrett Frey"
## [28] "Quentyn Martell"    "Samwell Tarly"      "Sansa Stark"
```

Example: Produce an integer vector that represents how many allegiances each character holds.

```
got_chars[[1]]$allegiances
```

```
## [1] "House Greyjoy of Pyke"
```

```
map_int(got_chars, function(x) length(x$allegiances))
```

```
##  [1] 1 1 1 0 1 0 0 1 1 2 1 1 2 2 0 1 3 2 1 1 1 2 1 1 1 0 1 1 1
```

Extracting elements

Handily, the map functions all allow the second argument to be an integer or string, and treat this internally as an appropriate extractor function

```
map_chr(got_chars, "name")
```

```
##  [1] "Theon Greyjoy"      "Tyrion Lannister"   "Victarion Greyj
##  [4] "Will"               "Areo Hotah"         "Chett"
##  [7] "Cressen"            "Arianne Martell"    "Daenerys Targar
## [10] "Davos Seaworth"     "Arya Stark"         "Arys Oakheart"
## [13] "Asha Greyjoy"       "Barristan Selmy"    "Varamyr"
## [16] "Brandon Stark"      "Brienne of Tarth"   "Catelyn Stark"
```

```
## [19] "Cersei Lannister"   "Eddard Stark"       "Jaime Lannister
## [22] "Jon Connington"     "Jon Snow"           "Aeron Greyjoy"
## [25] "Kevan Lannister"    "Melisandre"         "Merrett Frey"
```

```
map_lgl(got_chars, "alive")
```

```
##  [1]  TRUE  TRUE  TRUE FALSE  TRUE FALSE FALSE  TRUE  TRUE  TRU
## [13]  TRUE  TRUE FALSE  TRUE  TRUE FALSE  TRUE FALSE  TRUE  TRU
## [25] FALSE  TRUE FALSE FALSE  TRUE  TRUE
```

Interestingly, we can actually do the following in base R: `` `[`() `` and `` `[[`() `` are functions that act in the following way for an integer `x` and index `i`

- `` `[`(x, i) `` is equivalent to `x[i]`
- `` `[[`(x, i) `` is equivalent to `x[[i]]`

(This works whether `i` is an integer or a string)

```
sapply(got_chars, `[[`, "name")
```

```
##  [1] "Theon Greyjoy"      "Tyrion Lannister"   "Victarion Greyj
##  [4] "Will"               "Areo Hotah"         "Chett"
##  [7] "Cressen"            "Arianne Martell"    "Daenerys Targar
## [10] "Davos Seaworth"     "Arya Stark"         "Arys Oakheart"
## [13] "Asha Greyjoy"       "Barristan Selmy"    "Varamyr"
## [16] "Brandon Stark"      "Brienne of Tarth"   "Catelyn Stark"
## [19] "Cersei Lannister"   "Eddard Stark"       "Jaime Lannister
## [22] "Jon Connington"     "Jon Snow"           "Aeron Greyjoy"
## [25] "Kevan Lannister"    "Melisandre"         "Merrett Frey"
## [28] "Quentyn Martell"    "Samwell Tarly"      "Sansa Stark"
```

```
sapply(got_chars, `[[`, "alive")
```

```
##  [1]  TRUE  TRUE  TRUE FALSE  TRUE FALSE FALSE  TRUE  TRUE  TRU
## [13]  TRUE  TRUE FALSE  TRUE  TRUE FALSE  TRUE FALSE  TRUE  TRU
## [25] FALSE  TRUE FALSE FALSE  TRUE  TRUE
```

Part III: *A bit of* `dplyr`*:* `map_dfr()` *and* `map_dfc()`

`map_dfr()` and `map_dfc()`: list in, data frame out

The `map_dfr()` and `map_dfc()` functions iterate a function call over a list or vector, but automatically combine the results into a data frame. They differ in whether that data frame is formed by **r**ow-binding or **c**olumn-binding

```r
map_dfr(got_chars, `[`, c("name", "alive"))
```

```
## # A tibble: 30 × 2
##    name               alive
##    <chr>              <lgl>
##  1 Theon Greyjoy      TRUE
##  2 Tyrion Lannister   TRUE
##  3 Victarion Greyjoy  TRUE
##  4 Will               FALSE
##  5 Areo Hotah         TRUE
##  6 Chett              FALSE
##  7 Cressen            FALSE
##  8 Arianne Martell    TRUE
##  9 Daenerys Targaryen TRUE
## 10 Davos Seaworth     TRUE
## # ℹ 20 more rows
```

```r
# Base R is much less convenient
data.frame(name = sapply(got_chars, `[[`, "name"),
           alive = sapply(got_chars, `[[`, "alive"))
```

```
##                  name alive
## 1       Theon Greyjoy  TRUE
## 2    Tyrion Lannister  TRUE
## 3   Victarion Greyjoy  TRUE
## 4                Will FALSE
## 5          Areo Hotah  TRUE
## 6               Chett FALSE
## 7             Cressen FALSE
## 8     Arianne Martell  TRUE
## 9  Daenerys Targaryen  TRUE
## 10     Davos Seaworth  TRUE
## 11         Arya Stark  TRUE
## 12      Arys Oakheart FALSE
## 13       Asha Greyjoy  TRUE
## 14    Barristan Selmy  TRUE
## 15            Varamyr FALSE
## 16      Brandon Stark  TRUE
## 17   Brienne of Tarth  TRUE
## 18      Catelyn Stark FALSE
## 19   Cersei Lannister  TRUE
## 20       Eddard Stark FALSE
## 21    Jaime Lannister  TRUE
## 22     Jon Connington  TRUE
## 23           Jon Snow  TRUE
## 24      Aeron Greyjoy  TRUE
## 25    Kevan Lannister FALSE
## 26         Melisandre  TRUE
## 27       Merrett Frey FALSE
## 28    Quentyn Martell FALSE
## 29      Samwell Tarly  TRUE
## 30        Sansa Stark  TRUE
```

Note: the first example uses **extra arguments**; the map functions work just like the apply functions in this regard

`map_dfr()` requires matched names

Produce a matrix that has dimension 30 x 6, with each column representing a TV season, and each row a character. The matrix should have a value of `TRUE` in position (i,j) if character i was in season j, and `FALSE` otherwise.

```
got_chars[[1]]$tvSeries
```

```
## [1] "Season 1" "Season 2" "Season 3" "Season 4" "Season 5" "Sea
```

```
six_season <- c("Season 1", "Season 2", "Season 3", "Season 4", "S
```

Here, we introduce a useful operator `%in%` for matching, in particular, whether the elements in a vector `vec1` exist in another vector `vec2`. `vec1 %in% vec2` returns a logical vector with the same length as `vec1`. If there is a match in `vec2`, then return `TRUE`, otherwise return `FALSE`. A toy example:

```
1:5 %in% 3:6
```

```
## [1] FALSE FALSE  TRUE  TRUE  TRUE
```

```
map_dfr(got_chars, function(x) six_season %in% x$tvSeries)
```

```
## Error in `dplyr::bind_rows()`:
## ! Argument 1 must be a data frame or a named atomic vector.
```

```
map_dfr(got_chars, function(x){
  res <- six_season %in% x$tvSeries
  names(res) <- six_season
  return(res)
})
```

```
## # A tibble: 30 × 6
##    `Season 1` `Season 2` `Season 3` `Season 4` `Season 5` `Seas
##    <lgl>      <lgl>      <lgl>      <lgl>      <lgl>      <lgl>
##  1 TRUE       TRUE       TRUE       TRUE       TRUE       TRUE
##  2 TRUE       TRUE       TRUE       TRUE       TRUE       TRUE
##  3 FALSE      FALSE      FALSE      FALSE      FALSE      FALSE
```

```
##  4 FALSE      FALSE      FALSE      FALSE      FALSE      FALSE
##  5 FALSE      FALSE      FALSE      FALSE      TRUE       TRUE
##  6 FALSE      FALSE      FALSE      FALSE      FALSE      FALSE
##  7 FALSE      TRUE       FALSE      FALSE      FALSE      FALSE
##  8 FALSE      FALSE      FALSE      FALSE      FALSE      FALSE
##  9 TRUE       TRUE       TRUE       TRUE       TRUE       TRUE
## 10 FALSE      TRUE       TRUE       TRUE       TRUE       TRUE
```

`dplyr` package

The `map_dfr()` and `map_dfc()` functions actually belong to the `dplyr` package instead of `purrr`. `dplyr` is a member of the `tidyverse` package that is very useful for data frame computations. You'll learn more soon, but for now, you can think of it as providing the `tidyverse` alternative to the base R functions `subset()`, `split()`, `tapply()`

`filter()`: subset rows based on a condition

```
head(mtcars) # Built in data frame of cars data, 32 cars x 11 vari
```

```
##                    mpg cyl disp  hp drat    wt  qsec vs am gear
## Mazda RX4         21.0   6  160 110 3.90 2.620 16.46  0  1    4
## Mazda RX4 Wag     21.0   6  160 110 3.90 2.875 17.02  0  1    4
## Datsun 710        22.8   4  108  93 3.85 2.320 18.61  1  1    4
## Hornet 4 Drive    21.4   6  258 110 3.08 3.215 19.44  1  0    3
## Hornet Sportabout 18.7   8  360 175 3.15 3.440 17.02  0  0    3
## Valiant           18.1   6  225 105 2.76 3.460 20.22  1  0    3
```

```
filter(mtcars, (mpg >= 20 & disp >= 200) | (drat <= 3))
```

```
##                     mpg cyl disp  hp drat    wt  qsec vs am ge
## Hornet 4 Drive      21.4   6  258 110 3.08 3.215 19.44  1  0
## Valiant             18.1   6  225 105 2.76 3.460 20.22  1  0
## Cadillac Fleetwood  10.4   8  472 205 2.93 5.250 17.98  0  0
```

```
## Lincoln Continental 10.4   8  460 215 3.00 5.424 17.82  0  0
## Dodge Challenger    15.5   8  318 150 2.76 3.520 16.87  0  0
```

```
# Base R is just as easy with subset(), more complicated with dire
subset(mtcars, (mpg >= 20 & disp >= 200) | (drat <= 3))
```

```
##                      mpg cyl disp  hp drat    wt  qsec vs am ge
## Hornet 4 Drive      21.4   6  258 110 3.08 3.215 19.44  1  0
## Valiant             18.1   6  225 105 2.76 3.460 20.22  1  0
## Cadillac Fleetwood  10.4   8  472 205 2.93 5.250 17.98  0  0
## Lincoln Continental 10.4   8  460 215 3.00 5.424 17.82  0  0
## Dodge Challenger    15.5   8  318 150 2.76 3.520 16.87  0  0
```

```
mtcars[(mtcars$mpg >= 20 & mtcars$disp >= 200) | (mtcars$drat <= 3
```

```
##                      mpg cyl disp  hp drat    wt  qsec vs am ge
## Hornet 4 Drive      21.4   6  258 110 3.08 3.215 19.44  1  0
## Valiant             18.1   6  225 105 2.76 3.460 20.22  1  0
## Cadillac Fleetwood  10.4   8  472 205 2.93 5.250 17.98  0  0
## Lincoln Continental 10.4   8  460 215 3.00 5.424 17.82  0  0
## Dodge Challenger    15.5   8  318 150 2.76 3.520 16.87  0  0
```

`group_by()` : define groups of rows based on columns or conditions

```
group_by(mtcars, cyl)
```

```
## # A tibble: 32 × 11
## # Groups:   cyl [3]
##      mpg   cyl  disp    hp  drat    wt  qsec    vs    am  gear
##    <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
```

```
##  1  21       6  160    110  3.9   2.62  16.5     0     1     4
##  2  21       6  160    110  3.9   2.88  17.0     0     1     4
##  3  22.8     4  108     93  3.85  2.32  18.6     1     1     4
##  4  21.4     6  258    110  3.08  3.22  19.4     1     0     3
##  5  18.7     8  360    175  3.15  3.44  17.0     0     0     3
##  6  18.1     6  225    105  2.76  3.46  20.2     1     0     3
##  7  14.3     8  360    245  3.21  3.57  15.8     0     0     3
##  8  24.4     4  147.    62  3.69  3.19  20       1     0     4
##  9  22.8     4  141.    95  3.92  3.15  22.9     1     0     4
## 10  19.2     6  168.   123  3.92  3.44  18.3     1     0     4
```

- This doesn't actually change anything about the way the data frame looks
- Only difference is that when it prints, we're told about the groups
- But it will play a big role in how `dplyr` functions act on the data frame

`summarize()` : apply computations to (groups of) rows of a data frame

```r
# Ungrouped
summarize(mtcars, mpg = mean(mpg), hp = mean(hp))
```

```
##        mpg       hp
## 1 20.09062 146.6875
```

```r
# Grouped by number of cylinders
summarize(group_by(mtcars, cyl), mpg = mean(mpg), hp = mean(hp))
```

```
## # A tibble: 3 × 3
##     cyl   mpg    hp
##   <dbl> <dbl> <dbl>
## 1     4  26.7  82.6
## 2     6  19.7 122.
## 3     8  15.1 209.
```

Note: the use of `group_by()` makes the difference here

```r
# Base R, ungrouped calculation is not so bad
c("mpg" = mean(mtcars$mpg), "hp" = mean(mtcars$hp))
```

```
##       mpg        hp
##  20.09062 146.68750
```

```r
# Base R, grouped calculation is getting a bit ugly
cbind(tapply(mtcars$mpg, INDEX=mtcars$cyl, FUN=mean),
      tapply(mtcars$hp, INDEX=mtcars$cyl, FUN=mean))
```

```
##       [,1]      [,2]
## 4 26.66364  82.63636
## 6 19.74286 122.28571
## 8 15.10000 209.21429
```

```r
sapply(split(mtcars, mtcars$cyl), FUN=function(df) {
  return(c("mpg" = mean(df$mpg), "hp" = mean(df$hp)))
})
```

```
##            4         6        8
## mpg 26.66364  19.74286  15.1000
## hp  82.63636 122.28571 209.2143
```

```r
aggregate(mtcars[, c("mpg", "hp")], by=list(mtcars$cyl), mean)
```

```
##   Group.1      mpg        hp
## 1       4 26.66364  82.63636
## 2       6 19.74286 122.28571
## 3       8 15.10000 209.21429
```

Exercise

Using the `got_chars` dataset, create a data frame with three columns:

- `name` : Character name
- `num_book` : Number of books each character appears in using `books`

- `is_martell` : Logical (`TRUE` if the character belongs to the "House Martell" family based on `allegiances`)

▶ Solution

Summary

- For iteration tasks, `tidyverse` is an alternative of base R with better consistency
- `tidyverse` is a collection of packages for common data science tasks
- `purrr` is one such package that provides a consistent family of iteration functions
- `map()` : list in, list out
- `map_dbl()` , `map_lgl()` , `map_chr()` : list in, vector out (of a particular data type)
- `map_dfr()` , `map_dfc()` : list in, data frame out (row-binded or column-binded)
- `dplyr` is another such package that provides functions for data frame computations
- `filter()` : subset rows based on a condition
- `group_by()` : define groups of rows according to a condition
- `summarize()` : apply computations across groups of rows